

REPORTE DE AUDITORIA TECNICA

Evaluacion de Calidad - Proyecto proyecto-licorera

91 / 100 PUNTUACION GLOBAL	ESTADO: EXCELENTE / LISTO PARA ENTREGA Este reporte autoevalua el cumplimiento del proyecto frente a los 12 puntos de control de la rubrica tecnica de desarrollo (Frontend y Backend).
--	---

DETALLE DE EVALUACION

1. 1. Estructura de Componentes de Interfaz (Plantillas)	100 / 100
Excelente. La arquitectura de Django Templates usa herencia de plantillas de forma modular.	
Total de plantillas HTML detectadas: 31 Plantillas que heredan ({% extends %}): 22 Plantillas con fragmentos reusables ({% include %}): 3 Bloques definidos ({% block %}): 25 Uso de static ({% load static %}): 30	
2. 2. Enrutamiento de vistas y módulos	100 / 100
Enrutamiento correcto mediante urls.py centralizado y modular en Django.	
Archivos urls.py mapeados: 9 Rutas/endpoints totales definidos en el servidor: 106 - .\configuracion\urls.py - .\core\urls.py - .\inventario\urls.py - .\principal\urls.py	

- - .\productos\urls.py

- - .\proveedores\urls.py

- - .\reportes\urls.py

- - .\usuarios\urls.py

- - .\ventas\urls.py

3. 3. Consumo de API REST, carga y errores

100 / 100

Excelente. Se consumen APIs y se manejan correctamente los errores de conexión y estados de carga.

- - .\static\js\datatables.js (Frontend JS): 3 llamadas (Con manejo de errores y estado de carga)

- - .\static\js\datatables.min.js (Frontend JS): 1 llamadas (Con manejo de errores y estado de carga)

4. 4. Estilos y metodologías CSS (BEM / Atómico)

70 / 100

Diseño atómico basado en Bootstrap 5 / metodología BEM aplicado de forma general.

- Archivos CSS en la carpeta estática: 25

- Selectores con estructura BEM (___ o --) detectados: 470

- Estructuras de diseño atómico (clases utilitarias Bootstrap): 1421

- Estilos en línea (style='...') en HTML: 1188 (se sugiere minimizarlos)

5. 5. Binding de datos reactivo y DOM

100 / 100

Data binding unidireccional/bidireccional reactivo manejado mediante JS y listeners de eventos del DOM.

- Archivos JavaScript escaneados: 6

- Escrituras dinámicas en el DOM (binding de salida): 224

- Escuchadores de eventos de entrada (binding de entrada): 11

6. 6. Validación de formularios

100 / 100

Cumplido. Se realizan validaciones robustas tanto en backend (Django Forms) como en frontend.

- Formularios de Django estructurados (forms.py): 5
- Llamadas a validación segura en vistas (.is_valid()): 12
- Atributos de validación HTML5 en plantillas frontend: 93
- - .\productos\forms.py
- - .\proveedores\forms.py
- - .\reportes\forms.py
- - .\usuarios\forms.py
- - .\ventas\forms.py

7. 7. Configuración limpia por Variables de Entorno

50 / 100

Advertencia: Se detectan datos sensibles persistidos de forma directa en settings.py (ej. contraseñas de email o claves de desarrollo). Múdalos a un archivo .env.

- Archivo .env o .env.example presente: No (pero se detectó configuración de dotenv en código)
- Llamadas a variables de entorno en código (os.getenv/decouple): 5
- [ALERTA] Credencial expuesta: SECRET_KEY expuesta en settings.py
- [ALERTA] Credencial expuesta: EMAIL_HOST_PASSWORD expuesta en settings.py

8. 8. Jerarquía y organización del código fuente

100 / 100

Estructura de directorios altamente organizada bajo el estándar de aplicaciones Django MVT.

- - vistas (views.py): 8
- - modelos (models.py): 8
- - formularios (forms.py): 5
- - rutas (urls.py): 9
- - plantillas (.html): 31
- - recursos estaticos (.css/.js): 31

- - tests (tests.py): 8

9. 9. Pruebas unitarias sobre componentes

100 / 100

¡Excelente cobertura de pruebas! Se encontraron 184 métodos de prueba estructurados.

- Archivos de prueba detectados: 8

- Clases de Test definidos: 29

- Funciones de prueba (test_*): 184

- - .\configuracion\tests.py

- - .\inventariotests.py

- - .\principaltests.py

- - .\productos\tests.py

- - .\proveedores\tests.py

- - .\reportes\tests.py

- - .\usuarios\tests.py

- - .\ventas\tests.py

10. 10. Optimización de carga (Lazy loading / split)

75 / 100

Optimización parcial. Añade loading='lazy' en imágenes pesadas de tus catálogos o galerías.

- Imágenes o recursos con carga diferida (loading='lazy'): 2

- Uso de scripts asíncronos o diferidos (async/defer): 0

11. 11. Documentación (JSDoc / Comentarios)

100 / 100

Código fuente ampliamente documentado con estándares JSDoc y Docstrings descriptivos.

- Archivos de código escaneados: 79 Python, 6 JavaScript

- Docstrings de documentación en Python: 251

- Comentarios estilo JSDoc en JavaScript: 2761

12. 12. Adaptabilidad de la interfaz (Responsive)

100 / 100

Diseño completamente responsivo y adaptable mediante Bootstrap Grid y consultas de medios CSS.

- Media Queries detectadas en hojas de estilo CSS: 40
- Uso de contenedores y rejillas responsivas (Bootstrap Grid/Flexbox): 606

RECOMENDACIONES CLAVE PARA MEJORA:

- Reducir el uso de estilos en línea (`style="..."`) en las plantillas HTML (trasladar a CSS).
- Migrar las variables `SECRET_KEY` y `EMAIL_HOST_PASSWORD` a variables de entorno (`.env`).
- Añadir `loading="lazy"` a imágenes y `async/defer` a scripts.